Demonstration: getting set up with OpenRouter, open-weight models, and OpenCode

Rohan Alexander

7 October, 2026

Some models:

Family	Slug on OpenRouter	Input / output, USD per million tokens
DeepSeek V4 Flash	`deepseek/deepseek-v4-flash-0731`	0.06 / 0.18
DeepSeek V4 Pro	`deepseek/deepseek-v4-pro-0813`	1.32 / 3.96
GLM-5.3 Flash (Z.ai)	`z-ai/glm-5.3-flash`	0.15 / 0.50
GLM-5.3 (Z.ai)	`z-ai/glm-5.3`	1.40 / 4.40
Muse Glimmer 30B (Meta)	`meta/muse-glimmer-30b`	0.30 / 1.20
Gemma 4 31B (Google)	`google/gemma-4-31b-it:free`	Free
Gemma 4 26B A4B (Google)	`google/gemma-4-26b-a4b-it:free`	Free
Nemotron 3 Super 120B (NVIDIA)	`nvidia/nemotron-3-super-120b-a12b:free`	Free
Nemotron 3 Ultra 550B (NVIDIA)	`nvidia/nemotron-3-ultra-550b-a55b:free`	Free

Set up

OpenRouter account and key

1. Make an account at `https://openrouter.ai`.

2. Add ten dollars of credit.
3. Under Settings, then Keys, create a key and copy it. It starts with `sk-or-v1-`.

The key is like a combination of a username and password. If I have your key then I can spend your tokens. So don't put it in a script, or a notebook, or a Quarto document, or a repo. Put it in a file called `.env` in the root of the project, containing one line, and make sure that `.env` is in `.gitignore`:

```
echo 'OPENROUTER_API_KEY=sk-or-v1-...' > .env
echo ".env" >> .gitignore
```

The dots are your actual key, and there is a space after `echo`. If you would rather not type it in the shell, make the file in RStudio. File, then New File, then Text File. Paste the one line in. File, then Save As, name it `.env`, and save it in the project folder. RStudio warns that a name starting with a dot makes a hidden file, which is what you want. The Files pane hides it unless you tick Show Hidden Files under its gear menu. Every script then starts with `load_dotenv()`, which reads that file into the environment. It looks in the folder the script is in and then in each folder above it, so one `.env` at the root of a repo serves every script in it. If a GitHub Actions workflow needs the key, it goes in as a repository secret, like in Week 1 (De Angelis 2026). If you commit a key by accident, go to the OpenRouter page and revoke it, as a matter of priority.

A project

```
uv init silicon_samples
cd silicon_samples
uv add openai python-dotenv polars
```

We use the OpenAI Python package because OpenRouter speaks that API. The only differences are the base URL and the key.

A first call

Save this as `00-hello.py` and run it with `uv run 00-hello.py`.

```
#### Preamble ####
# Purpose: First call to a model through OpenRouter; print the answer and the cost
# Author: Rohan Alexander
# Date: 7 October 2026
# Contact: rohan.alexander@utoronto.ca
```

```python
# License: MIT
# Pre-requisites: uv add openai python-dotenv; OPENROUTER_API_KEY in .env

#### Workspace setup ####
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],
)

#### Call ####
MODEL = "deepseek/deepseek-v4-flash-0731"

response = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "user", "content": "In one sentence, what is a silicon sample?"}
    ],
    temperature=0.7,
    max_tokens=2000,
)

print(response.choices[0].message.content)
print(response.usage)
```

Three things to look at in the output. The text. The `usage` object, which OpenRouter always returns and which includes `prompt_tokens`, `completion_tokens`, and `cost` in US dollars. And `response.model`, which tells you which checkpoint actually answered; with an alias it can differ from what you asked for.

Now change `MODEL` to `z-ai/glm-5.3-flash` and then to `meta/muse-glimmer-30b` and run it again.

One thing to know about `max_tokens`. All three of these models think before they answer, and the thinking counts against the limit. If the limit is too small then the model spends all of it thinking, the content comes back as `None`, and you still pay. That is why the limit above is 2,000 for a one-sentence answer. You can see how much was spent on thinking in `reasoning_tokens` in

the usage object, and you can ask for less of it with `extra_body={"reasoning": {"effort": "low"}}`, but you cannot turn it off for every model, so give the limit room.

Structured outputs

A silicon respondent has to give an answer we can put in a data frame ideally. We can ask for JSON that matches a schema, and we tell the model to be strict about it. Save this as `01-structured.py` and run it.

```
#### Preamble ####
# Purpose: One persona answers a vote-choice question with a structured output
# Author: Rohan Alexander
# Date: 7 October 2026
# Pre-requisites: as for 00-hello.py

import json
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(base_url="https://openrouter.ai/api/v1", api_key=os.environ["OPENROUTER_API_

MODEL = "deepseek/deepseek-v4-flash-0731"

persona = (
    "You are a 34-year-old woman who rents an apartment in Toronto's Ward 13, "
    "works in retail, finished high school, and did not vote in the 2022 municipal election."
)

question = (
    "The Toronto mayoral election is on Monday 26 October 2026. "
    "If it were held today, which candidate would you vote for: Olivia Chow, Brad Bradford, "
    "Chris Alexander, someone else, or would you not vote? "
    "How confident are you, from 0 to 100?"
)

schema = {
    "type": "object",
    "properties": {
        "vote": {
```

```
            "type": "string",
            "enum": ["Chow", "Bradford", "Alexander", "Other", "Would not vote"],
        },
        "confidence": {"type": "integer", "minimum": 0, "maximum": 100},
    },
    "required": ["vote", "confidence"],
    "additionalProperties": False,
}

response = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": persona},
        {"role": "user", "content": question},
    ],
    temperature=1.0,
    response_format={
        "type": "json_schema",
        "json_schema": {"name": "vote_choice", "strict": True, "schema": schema},
    },
)

answer = json.loads(response.choices[0].message.content)
print(answer, response.usage.cost)
```

Two things to notice. First, `temperature`. At 0 the model gives nearly the same answer every time, which is not what a sample of people does. Run the script five times at temperature 0 and five times at 1.0 and look at the two sets of answers. Second, not every model on every provider enforces `strict`. If the JSON fails to parse, that is data too: record it and move on, do not retry silently.

Batching, caching, and cost

Three hundred personas times three models is 900 calls. Three suggestions, and then a script that does all three for twelve personas. Save it as `02-batch.py` and run it twice. The second run costs nothing.

1. Try to not call the model twice for the same (model, prompt, temperature, seed). Save every response as it arrives, keyed on a hash of those four things, and check the cache before calling. A crash at call 700 then costs nothing to resume.

2. Add up `usage.cost` as you go and stop when you hit a budget you set at the top of the script.
3. Run calls concurrently but politely. Eight at a time is plenty. OpenRouter will return a 429 if you go too fast.

```
#### Preamble ####
# Purpose: Ask many personas the same question across several models, with caching and a budg
# Author: Rohan Alexander
# Date: 7 October 2026
# Pre-requisites: uv add openai python-dotenv polars numpy

import hashlib
import json
import os
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

import numpy as np
import polars as pl
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(base_url="https://openrouter.ai/api/v1", api_key=os.environ["OPENROUTER_API_K

MODELS = ["deepseek/deepseek-v4-flash-0731", "z-ai/glm-5.3-flash", "meta/muse-glimmer-30b"]
TEMPERATURE = 1.0
BUDGET_USD = 2.00
CACHE = Path("cache")
CACHE.mkdir(exist_ok=True)

#### Personas ####
# Twelve Toronto voters from a few marginals, drawn independently, with a fixed seed
rng = np.random.default_rng(seed=853)
N = 12
age = rng.choice(["in your twenties", "in your thirties", "in your forties", "in your fifties
gender = rng.choice(["a woman", "a man"], size=N)
tenure = rng.choice(["rent", "own your home"], size=N, p=[0.47, 0.53])
area = rng.choice(["downtown", "Scarborough", "North York", "Etobicoke", "East York"], size=N
personas = [
    f"You are {g} {a} who lives in {ar}, Toronto, and you {t}."
    for g, a, ar, t in zip(gender, age, area, tenure)
]
```

```
personas = list(dict.fromkeys(personas))  # drop any duplicates

#### Question ####
question = (
    "The Toronto mayoral election is on Monday 26 October 2026. "
    "If it were held today, which candidate would you vote for: Olivia Chow, Brad Bradford, "
    "Chris Alexander, someone else, or would you not vote? "
    "How confident are you, from 0 to 100?"
)
schema = {
    "type": "object",
    "properties": {
        "vote": {"type": "string", "enum": ["Chow", "Bradford", "Alexander", "Other", "Would
        "confidence": {"type": "integer", "minimum": 0, "maximum": 100},
    },
    "required": ["vote", "confidence"],
    "additionalProperties": False,
}

#### Calls ####
spent = 0.0


def key(model, persona):
    raw = f"{model}|{TEMPERATURE}|{persona}|{question}"
    return hashlib.sha256(raw.encode()).hexdigest()


def ask(model, persona):
    global spent
    path = CACHE / f"{key(model, persona)}.json"
    if path.exists():
        return json.loads(path.read_text())
    if spent >= BUDGET_USD:
        raise RuntimeError(f"Budget of ${BUDGET_USD} reached")
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "system", "content": persona}, {"role": "user", "content": questio
        temperature=TEMPERATURE,
        response_format={"type": "json_schema", "json_schema": {"name": "vote_choice", "stric
    )
    spent += response.usage.cost
```

```
    record = {
        "model_requested": model,
        "model_answered": response.model,
        "persona": persona,
        "raw": response.choices[0].message.content,
        "cost": response.usage.cost,
    }
    path.write_text(json.dumps(record))
    return record


jobs = [(m, p) for m in MODELS for p in personas]
with ThreadPoolExecutor(max_workers=8) as pool:
    records = list(pool.map(lambda mp: ask(*mp), jobs))

#### Results ####
for r in records:
    try:
        answer = json.loads(r["raw"])
        r["vote"], r["confidence"] = answer["vote"], answer["confidence"]
    except (TypeError, ValueError, KeyError):
        r["vote"], r["confidence"] = None, None  # no answer, or not the shape we asked for

df = pl.DataFrame(records)
df.write_parquet("silicon_sample.parquet")
print(df.group_by("model_requested", "vote").len().sort("model_requested", "vote"))
print("Unusable answers:", df["vote"].null_count())
print(f"Spent ${spent:.4f} this run")
```

OpenCode

OpenCode is an open-source coding agent that runs in the terminal. It is a harness like Claude Code and Codex, but it will use any model, including the open-weight ones above through OpenRouter.

Install it:

```
curl -fsSL https://opencode.ai/install | bash
```

or `brew install anomalyco/tap/opencode`. Then, inside the project folder:

```
opencode
```

Inside OpenCode, type `/connect`, choose OpenRouter, and paste your key. Then `/models` lists what is available; pick one of the three above. To pin the choice for the project, so that everyone on the team gets the same model, put an `opencode.json` at the root of the repo:

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "openrouter/deepseek/deepseek-v4-flash-0731",
  "provider": {
    "openrouter": {
      "models": {
        "deepseek/deepseek-v4-flash-0731": {},
        "z-ai/glm-5.3-flash": {},
        "meta/muse-glimmer-30b": {}
      }
    }
  }
}
```

Model names in OpenCode are `openrouter/` followed by the OpenRouter slug. Commit `opencode.json`. The key lives in `~/.local/share/opencode/auth.json`, which is outside the repo.

Run `/init` to read the project and write an `AGENTS.md`, which is where you put the things you would tell a new research assistant: use `uv run`, use polars not pandas, write parquet, put the preamble at the top of every script. Commit that too.

Live demonstration: open the Week 1 shelter project and ask OpenCode to "add a script `05-test.py` that checks `cleaned_shelter_usage.parquet` has no negative usage and no date after today, and fails loudly if it does". Watch it read the files, propose a diff, and run the script. Then switch the model with `/models` and ask for the same thing.

Rules for using it in this course.

- Read every diff before accepting it. You are the author of the commit.
- Never let it commit or push for you. That is something you should do.
- Do not give it your `.env`. It does not need the key to write code that uses the key.
- Disclose it in a detailed AI-use statement.

References

De Angelis, Inessa. 2026. *Using GitHub Actions for Computational Communication Research.* SocArXiv. https://doi.org/10.31235/osf.io/uqf6n_v1.